

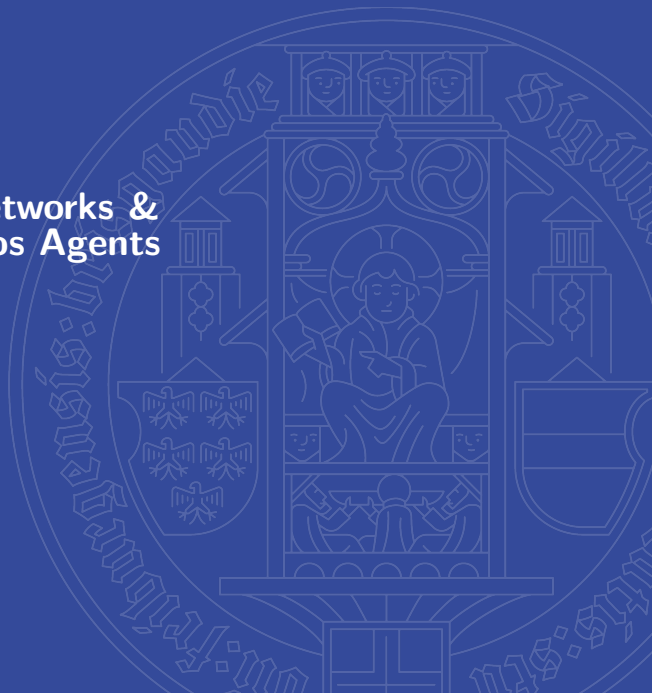
Agent Modeling with Neural Networks & Environments with Homogenous Agents

Multiagent Reinforcement Learning Seminar

Mahdi Gheidi
Seong Jin You

Neurorobotics Group
Department of Computer Science
University of Freiburg

July 15th 2026



Outline

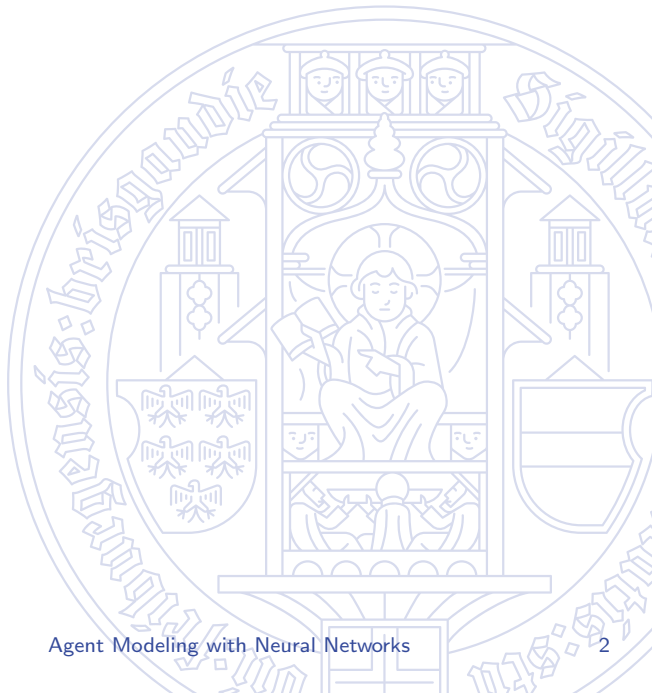
Agent Modeling with Neural Networks

Joint action learning

Representation Learning

Environments with Homogeneous Agents

Summary



Why Agent Modeling?

How do previous MARL methods handle other agents?

Independent Learning

Treat others as part of the environment.

e.g. opponent behavior affects reward and next state

Centralized Critic

Uses joint observations/actions during training.

e.g. COMA critic conditions on the joint action

Value Decomposition

Uses centralized training information.

e.g. global state in QMIX

Still no explicit modeling of other agents' policies

Agent Modeling: Learn an explicit model of other agents' policies.

From Tabular JAL-AM to Deep JAL-AM

Tabular JAL-AM

State

Representation

Lookup Table

$$A = 8, B = 2$$



$$\hat{\pi}(A | s) = 0.8$$

$$\hat{\pi}(B | s) = 0.2$$



Cannot generalize to unseen states

Deep JAL-AM

Observation History

Representation

Neural Network

Learned mapping



$$\hat{\pi}(A | h) = 0.8$$

$$\hat{\pi}(B | h) = 0.2$$

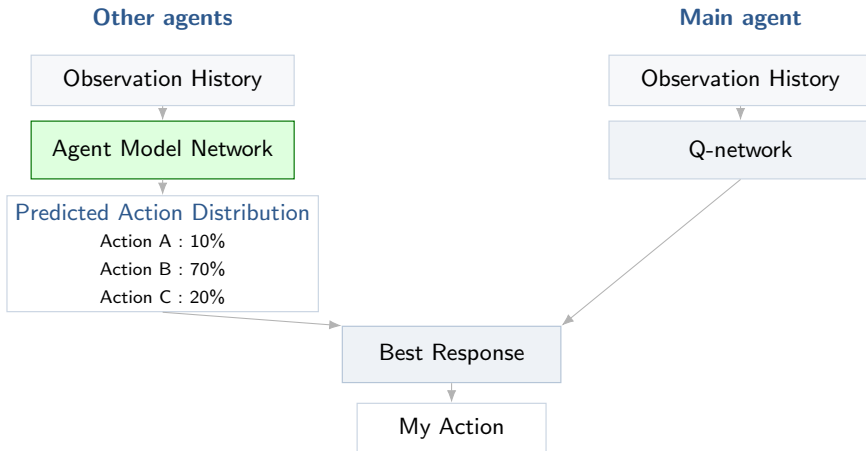


Generalizes to unseen observations

Deep JAL-AM Overview

Challenge: Decentralized execution → Cannot observe other agents' actions

Idea: Predict other agents' policies



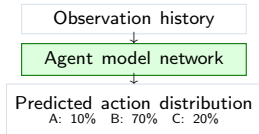
Training Deep JAL-AM

Algorithm Steps 1 & 2

Algorithm 23 Deep joint-action learning

```
1: Initialize  $n$  value networks with random parameters  $\theta_1, \dots, \theta_n$ 
2: Initialize  $n$  target networks with parameters  $\bar{\theta}_1, \dots, \bar{\theta}_n = \theta_n$ 
3: Initialize  $n$  policy networks with parameters  $\phi_{-1}^1, \dots, \phi_{-n}^n$ 
4: Initialize a replay buffer for each agent  $D_1, D_2, \dots, D_n$ 
5: for time step  $t = 0, 1, 2, \dots$  do
6:   Collect current observations  $o_1^t, \dots, o_n^t$ 
7:   for agent  $i = 1, \dots, n$  do
8:     With probability  $\epsilon$ : choose random action  $a_i^t$ 
9:     Otherwise: choose  $a_i^t \in \arg \max_{a_i} AV(h_i, a_i; \theta_i)$ 
10:  Apply actions  $(a_1^t, \dots, a_n^t)$ ; collect rewards  $r_1^t, \dots, r_n^t$  and next observations  $o_1^{t+1}, \dots, o_n^{t+1}$ 
11:  for agent  $i = 1, \dots, n$  do
12:    Store transition  $(h_i^t, a_i^t, r_i^t, h_i^{t+1})$  in replay buffers  $D_i$ 
13:    Sample random mini-batch of  $B$  transitions  $(h_i^k, a_i^k, r_i^k, h_i^{k+1})$  from  $D_i$ 
14:    if  $s^{k+1}$  is terminal then
15:      Targets  $y_i^k \leftarrow r_i^k$ 
16:    else
17:      Targets  $y_i^k \leftarrow r_i^k + \gamma \max_{a_i' \in A_i} AV(h_i^{k+1}, a_i'; \bar{\theta}_i)$ 
18:  4 Critic loss  $\mathcal{L}(\theta_i) \leftarrow \frac{1}{B} \sum_{k=1}^B \left( y_i^k - Q(h_i^k, \langle a_i^k, a_{-i}^k \rangle; \theta_i) \right)^2$ 
19:  3 Model loss  $\mathcal{L}(\phi_{-i}^i) = \sum_{j \neq i} \frac{1}{B} \sum_{k=1}^B -\log \hat{\pi}_j^i(a_j^k | h_i^k; \phi_j^i)$ 
20:  Update parameters  $\theta_i$  by minimizing the loss  $\mathcal{L}(\theta_i)$ 
21:  Update parameters  $\phi_{-i}^i$  by minimizing the loss  $\mathcal{L}(\phi_{-i}^i)$ 
22:  In a set interval, update target network parameters  $\bar{\theta}_i$ 
```

1. Agent Model Network



Predicts an action distribution for each other agent.

2. Replay Buffer

$D_1, \dots, D_n$
Each D_i stores transitions from agent i 's perspective:
 (h_i, a_i, r_i, h_i')

History h_i → input
Other-agent action a_j → ground-truth label
Observed actions from sampled transitions become labels for training the agent model.

Training Deep JAL-AM

Algorithm Steps 3 & 4

Algorithm 23 Deep joint-action learning

```
1: Initialize  $n$  value networks with random parameters  $\theta_1, \dots, \theta_n$ 
2: Initialize  $n$  target networks with parameters  $\bar{\theta}_1 = \theta_1, \dots, \bar{\theta}_n = \theta_n$ 
3: Initialize  $n$  policy networks with parameters  $\phi_{-1}^1, \dots, \phi_{-n}^n$ 
4: Initialize a replay buffer for each agent  $D_1, D_2, \dots, D_n$ 
5: for time step  $t = 0, 1, 2, \dots$  do
6:   Collect current observations  $o_1^t, \dots, o_n^t$ 
7:   for agent  $i = 1, \dots, n$  do
8:     With probability  $\epsilon$ : choose random action  $a_i^t$ 
9:     Otherwise: choose  $a_i^t \in \arg \max_{a_i} AV(h_i, a_i; \theta_i)$ 
10:  Apply actions  $(a_1^t, \dots, a_n^t)$ ; collect rewards  $r_1^t, \dots, r_n^t$  and next observations  $o_1^{t+1}, \dots, o_n^{t+1}$ 
11:  for agent  $i = 1, \dots, n$  do
12:    Store transition  $(h_i^t, a_i^t, r_i^t, h_i^{t+1})$  in replay buffers  $D_i$ 
13:    Sample random mini-batch of  $B$  transitions  $(h_i^k, a_i^k, r_i^k, h_i^{k+1})$  from  $D_i$ 
14:    if  $s^{k+1}$  is terminal then
15:      Targets  $y_i^k \leftarrow r_i^k$ 
16:    else
17:      Targets  $y_i^k \leftarrow r_i^k + \gamma \max_{a_i' \in A_i} AV(h_i^{k+1}, a_i'; \bar{\theta}_i)$ 
18:  Critic loss  $\mathcal{L}(\theta_i) \leftarrow \frac{1}{B} \sum_{k=1}^B \left( y_i^k - Q(h_i^k, \langle a_i^k, a_{-i}^k \rangle; \theta_i) \right)^2$ 
19:  Model loss  $\mathcal{L}(\phi_{-i}^i) = \sum_{j \neq i} \frac{1}{B} \sum_{k=1}^B -\log \hat{\pi}_j^i(a_j^k | h_i^k; \phi_j^i)$ 
20:  Update parameters  $\theta_i$  by minimizing the loss  $\mathcal{L}(\theta_i)$ 
21:  Update parameters  $\phi_{-i}^i$  by minimizing the loss  $\mathcal{L}(\phi_{-i}^i)$ 
22:  In a set interval, update target network parameters  $\bar{\theta}_i$ 
```

3. Agent Model Loss

$$\mathcal{L}(\phi_j^i) = -\log \hat{\pi}_j^i(a_j^t | h_i^t; \phi_j^i)$$

Prediction

A: 0.60

B: 0.30

C: 0.10

Ground Truth

Observed action: **B**

Cross-entropy **penalizes** low probability assigned to the observed action.

4. Joint-Action Value Function

$$\mathcal{L}(\theta_i) = \frac{1}{B} \sum_{(h_i^t, a_i^t, r_i^t, h_i^{t+1}) \in B} \left(r_i^t + \gamma \max_{a_i' \in A_i} AV(h_i^{t+1}, a_i'; \bar{\theta}_i) - Q(h_i^t, \langle a_i^t, a_{-i}^t \rangle; \theta_i) \right)^2$$

- DQN-style **temporal-difference** objective
- Uses predicted action distributions
- Updates centralized Q-network

Sampled Action Value

Exact Action Value

$$AV(h_i, a_i; \theta_i) = \sum_{a_{-i} \in A_{-i}} Q(h_i, \langle a_i, a_{-i} \rangle; \theta_i) \hat{\pi}_{-i}^i(a_{-i} \mid h_i; \phi_{-i}^i)$$

$$= \sum_{a_{-i} \in A_{-i}} Q(h_i, \langle a_i, a_{-i} \rangle; \theta_i) \prod_{j \neq i} \hat{\pi}_j^i(a_j \mid h_i; \phi_j^i)$$

Uses every possible action combination.

Computationally expensive



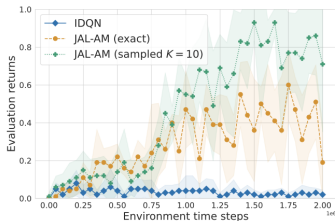
Sampled Action Value

$$AV(h_i, a_i; \theta_i) = \frac{1}{K} \sum_{k=1}^K Q(h_i, \langle a_i, a_{-i}^k \rangle; \theta_i), \quad a_j^k \sim \hat{\pi}_j^i(\cdot \mid h_i)$$

Approximate the expectation

using only K sampled joint actions.

Performance Comparison



Prediction: A: 90% B: 5% C: 5%

Exact AV

$0.90 Q(A) + 0.05 Q(B) + 0.05 Q(C)$

Uses all action values.

Sampled AV ($K = 10$)

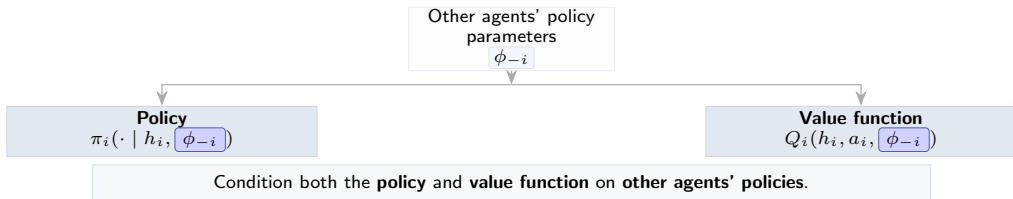
A A A A A A A A B A
 $0.90 Q(A) + 0.10 Q(B)$

Rare action C is absent in this update.

Possible explanation: Sampling noise may encourage more diverse action selection.

Why Representation Learning?

Ideal Goal



Problem

Unavailable during decentralized execution

Other agents' policies cannot be directly observed during decentralized execution.

Too high-dimensional to condition on

$$\pi_i(\cdot \mid h_i, \phi_{-i})$$

$$\phi_{-i} = [\phi_1, \dots, \phi_{i-1}, \phi_{i+1}, \dots, \phi_n]$$

Very large conditioning input

Continuously evolving policies

Other agents are learning, so their policies are non-stationary.

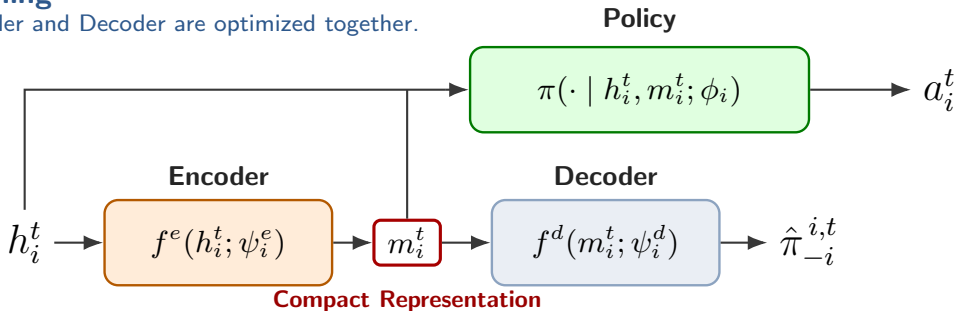
$$\pi_j(a_j \mid h_j; \phi_j^t) \rightarrow \pi_j(a_j \mid h_j; \phi_j^{t+1})$$

Can we learn a **compact representation** instead?

Encoder–Decoder Architecture

Training

Encoder and Decoder are optimized together.



Encoder

- Compresses the observation history
- Learns the compact embedding m_i^t

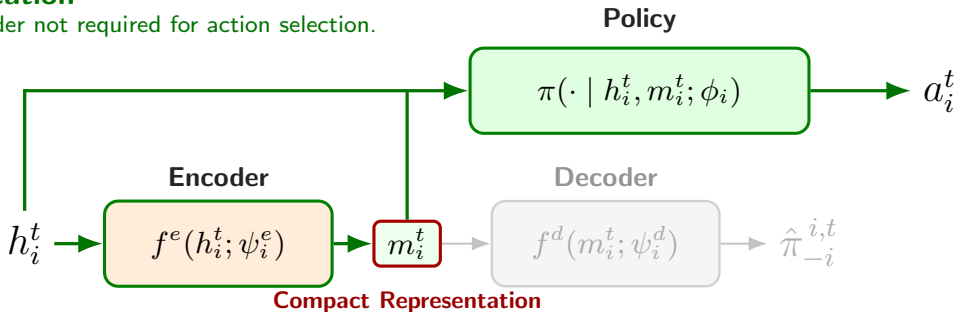
Decoder

- Reconstructs other agents' policy distributions
- Provides the reconstruction training signal

Encoder–Decoder Architecture

Execution

Decoder not required for action selection.



Encoder

- Compresses the observation history
- Learns the compact embedding m_i^t

Decoder

- Reconstructs other agents' policy distributions
- Provides the reconstruction training signal

Applying Representation Learning to A2C

Representation-based approach

Representation-based approach



General framework

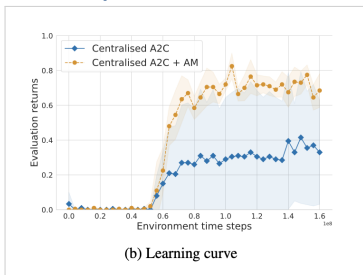
A2C + Agent Modeling

Policy: $\pi(\cdot \mid h_i, \mathbf{m}_i; \phi_i)$

Value function: $V(h_i, z, \mathbf{m}_i; \theta_i)$

Both the actor and critic use the learned agent representation, without modifying the overall A2C framework.

Experimental Result



With Agent Modeling

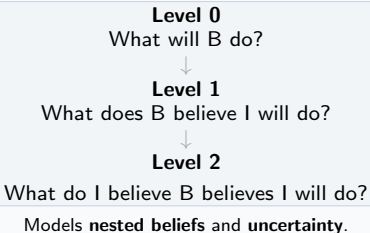
- Learns faster
- Higher final return
- More consistent learning
Lower variance across runs.

Advanced Agent Modeling

Beyond behavior prediction

1. Probabilistic Recursive Reasoning

How will the other agent react?



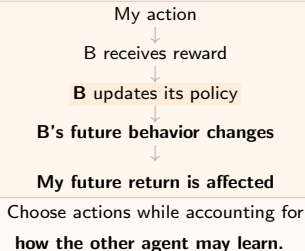
Prior → Observation → **Posterior**

Recursive reasoning:

Predict how others will react.

2. Opponent Shaping

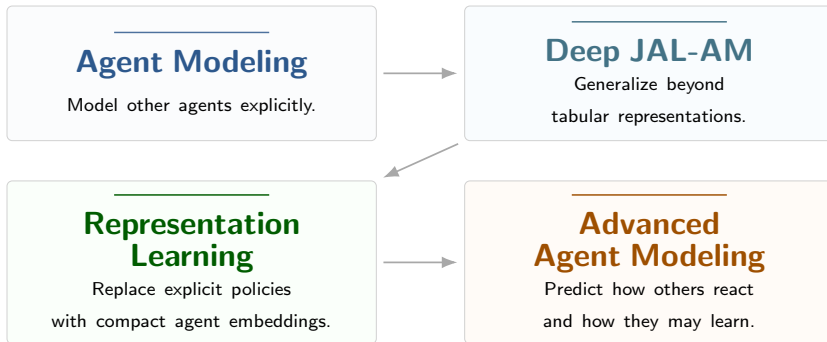
How will my action influence
their future learning?



Opponent shaping:

Influence how others will learn.

Agent Modeling: Key Takeaways



Explicit models of other agents enable **better decision-making** in multiagent environments.

Outline

Agent Modeling with Neural Networks

Environments with Homogeneous Agents

- Homogeneous Agents

- Parameter Sharing

- Experience Sharing

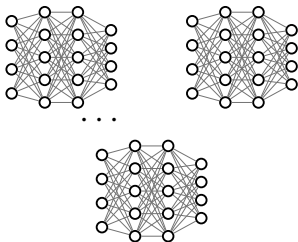
Summary



Homogeneous agents: Motivation

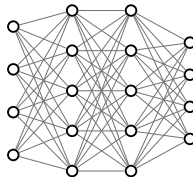
Previously, we have seen environments and agents in which we have used independent learning algorithms such as IDQN. Problem? The huge number of parameters and the slow training process.

Multiple independent agents



Each of the K agents has its own network
 $K \times N$ parameters

Shared network

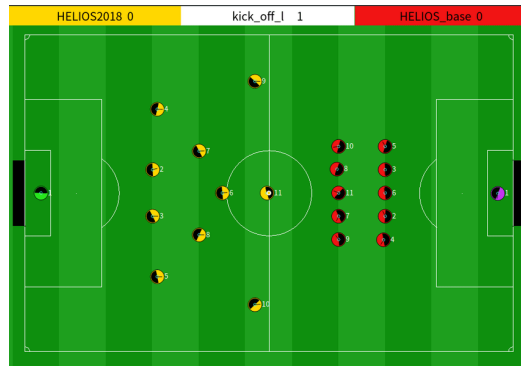


All agents use the same network
 N parameters

Homogeneous Agents: Motivation



Starcraft: a game with Agents with different roles, and usually different goals



Robocup Simulation: The agents have the same action space, same goals and observation abilities, and are interchangeable

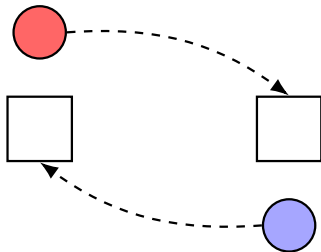
Weakly Homogeneous Agents

Definition: Weakly homogeneous agents

An environment has weakly homogeneous agents if for any joint policy $\pi = (\pi_1, \pi_2, \dots, \pi_n)$ and permutation between agents $\sigma : I \rightarrow I$, the following holds:

$$U_i(\pi) = U_{\sigma(i)}(\langle \pi_{\sigma(1)}, \pi_{\sigma(2)}, \dots, \pi_{\sigma(n)} \rangle), \quad \forall i \in I$$

Intuition: agents can *swap* their policies without changing the expected returns — the agents are interchangeable.



Agents have to move to two *different* locations.

Strongly Homogeneous Agents

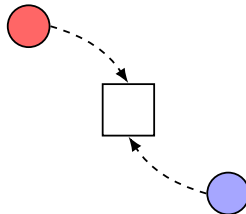
But what if agents are weakly homogeneous and even the goals are the same?

Definition: Strongly homogeneous agents

An environment has strongly homogeneous agents if the optimal joint policy consists of identical individual policies, formally:

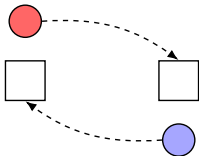
- The environment has weakly homogeneous agents.
- The optimal joint policy $\pi^* = (\pi_1, \pi_2, \dots, \pi_n)$ consists of identical policies $\pi_1 = \pi_2 = \dots = \pi_n$

Intuition: not only are the agents interchangeable — at the optimum they all *behave identically*.



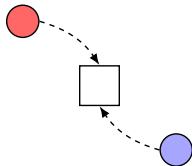
Both agents have to move to the *same* location.

Homogeneous vs. Non-Homogeneous Agents



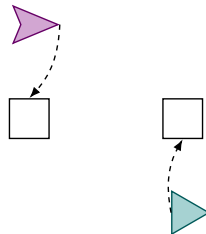
(a) Weakly homogeneous agents

Agents are interchangeable, but must move to different locations.



(b) Strongly homogeneous agents

Both agents can move to the same location.



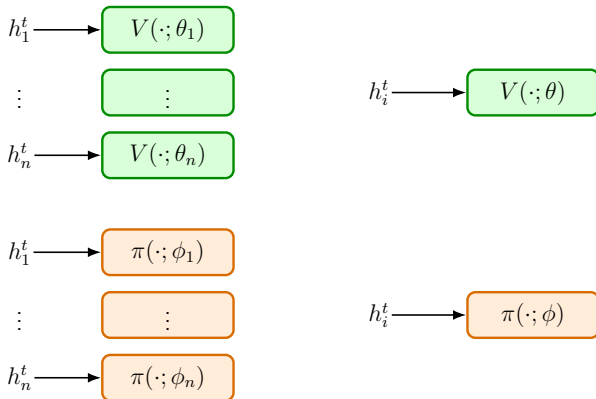
(c) Non-homogeneous agents

Agents are inherently different: different roles, capabilities, or action spaces.

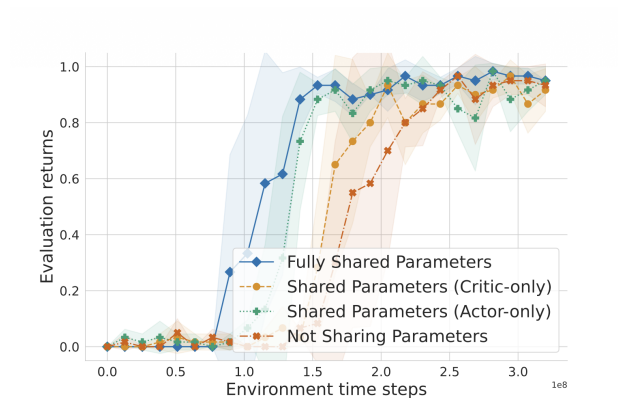
But if we have homogeneity, how can we use this for our gain?

Parameter sharing

When having strongly homogeneous agents, we know that all agents use identical policies. So do we need to keep different parameters for each?



Parameter sharing in LBF, a comparison



Learning curves for independent A2C with and without parameter sharing on level-based foraging. Figure taken from [ACS24].

Parameter Sharing: A Free Lunch?

Pros

- Number of parameters stays **constant** in the number of agents $\Rightarrow$ less compute, faster training
- Shared parameters are updated with the experience of **all** agents $\Rightarrow$ more diverse data, better sample-efficiency

Cons

- Relies on **strongly** homogeneous agents — a strong assumption that is hard to verify, and often simply not true
- Forces **identical** policies: with only weakly homogeneous agents, the optimal joint policy cannot even be represented

$\Rightarrow$ Can we keep the benefits *without* forcing identical policies?

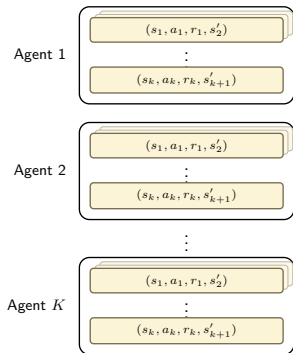
Idea: keep separate parameters, but share the *experience*.

Experience sharing

So, how do we exploit homogeneity properties in environments with weakly homogeneous agents?
Agents may need to learn different policies but that doesn't mean that they cannot use experiences of each other.
How?

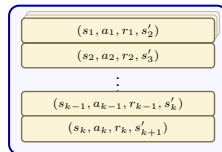
Independent vs. Shared Replay Buffers

Independent replay buffers



Each agent stores transitions
in its own replay buffer

Shared replay buffer



All agents store transitions
in one shared replay buffer

Caveat of experience sharing

- The shared replay buffer works because DQN is **off-policy**: it can learn from transitions generated by *any* policy — including other agents' policies.
- **On-policy** algorithms such as A2C or PPO require trajectories sampled from the *current* policy being optimized $\Rightarrow$ we cannot directly feed them other agents' experience.

So, is experience sharing off-limits for on-policy methods?

No — Shared Experience Actor-Critic (SEAC) uses *importance sampling* to correct for transitions collected by other agents.

Recap: Importance Sampling [SB18]

Estimating expectations under a different distribution

We can estimate an expectation under a *target* policy π using samples generated by a *behavior* policy π_β by re-weighting each sample with the **importance sampling ratio**:

$$\mathbb{E}_{a \sim \pi}[f(a)] = \mathbb{E}_{a \sim \pi_\beta} \left[\frac{\pi(a)}{\pi_\beta(a)} f(a) \right]$$

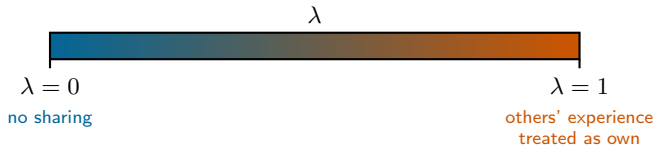
Applied to the policy gradient, the loss for off-policy optimization from a behavioral policy π_β becomes:

$$\mathcal{L}(\phi) = -\frac{\pi(a^t|h^t;\phi)}{\pi_\beta(a^t|h^t)} (r^t + \gamma V(h^{t+1};\theta) - V(h^t;\theta)) \log \pi(a^t | h^t; \phi)$$

⇒ Exactly the correction SEAC needs to reuse *other agents'* on-policy trajectories.

$$\mathcal{L}(\phi_i) = - (r_i^t + \gamma V(h_i^{t+1}; \theta_i) - V(h_i^t; \theta_i)) \log \pi(a_i^t | h_i^t; \phi_i) \\ - \lambda \sum_{k \neq i} \underbrace{\frac{\pi(a_k^t | h_k^t; \phi_i)}{\pi(a_k^t | h_k^t; \phi_k)}}_{\text{importance sampling}} (r_k^t + \gamma V(h_k^{t+1}; \theta_i) - V(h_k^t; \theta_i)) \log \pi(a_k^t | h_k^t; \phi_i)$$

- **Own experience:** the standard on-policy actor-critic loss of agent i .
- **Shared experience:** trajectories of all other agents $k \neq i$, treated as off-policy data and corrected via importance sampling — this is what makes experience sharing possible in an *on-policy* setting.



Outline

Agent Modeling with Neural Networks

Environments with Homogeneous Agents

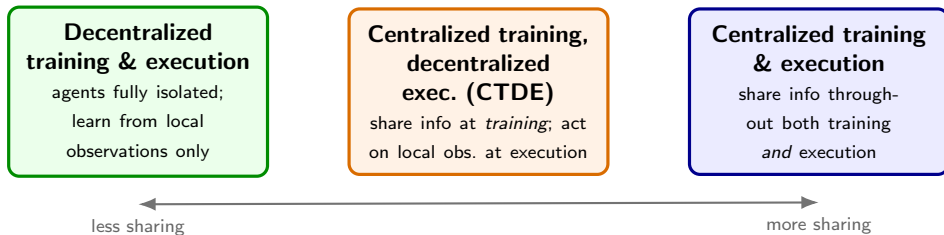
Summary



Chapter 9 — The Big Picture

Training & execution paradigms

Across all of Chapter 9, algorithms differ in *what information is shared, and when*. **Centralized information** is anything beyond an agent's own observation — parameters, gradients, observations, or actions of others.



CTDE is the popular middle ground: exploit centralized information to *learn* better, keep policies deployable with only local information.

Chapter 9 — Four Recurring Challenges

Deep MARL is largely the story of *four* core challenges — each section attacks one or more of them:

Non-stationarity

Others learn and change $\Rightarrow$ a moving target.
Centralized critics (9.4), agent modeling (9.6).

Multi-agent credit assignment

Who caused the shared reward? *Value decomposition — VDN, QMIX (9.5).*

Scaling & sample efficiency

Cost grows with the number of agents. *Parameter & experience sharing (9.7).*

Equilibrium selection

Which joint policy to converge to? *Self-play (9.8), population-based training (9.9).*

Partial observability cuts across all four — handled throughout via observation histories h_i^t and centralized information z^t .

What did we discuss in Chapter 9?

Section	Technique	Challenge addressed
9.3 Independent Learning	Treat others as environment	Baseline — exposes non-stationarity
9.4 Policy Gradients	Centralized critics	Credit assignment, non-stationarity (training)
9.5 Value Decomposition	VDN, QMIX	Multi-agent credit assignment, at scale
9.6 Agent Modeling	Deep JAL-AM, representations	Reason about & adapt to other agents; generalize to unseen states
9.7 Homogeneous Agents	Parameter & experience sharing	Scaling, sample efficiency
9.8 Self-Play	MCTS, AlphaZero	Equilibrium-finding, 2-player zero-sum
9.9 Population-Based Training	PSRO, AlphaStar	Equilibrium-finding & robustness, general competitive

All build on the same paradigms (decentralized / CTDE / centralized) — they differ in *what* is shared and *which* challenge it targets.

Reference

- [ACS24] Stefano V Albrecht, Filippos Christianos, and Lukas Schäfer.
Multi-agent reinforcement learning: Foundations and modern approaches.
MIT Press, 2024.
- [CSA20] Filippos Christianos, Lukas Schäfer, and Stefano Albrecht.
Shared experience actor-critic for multi-agent reinforcement learning.
Advances in neural information processing systems, 33:10707–10717, 2020.
- [SB18] Richard S Sutton and Andrew G Barto.
Reinforcement learning: An introduction.
MIT press, 2018.